



Universidade Federal do Cariri
Ciência da Computação
Juazeiro do Norte, CE
2025

Computação Gráfica

Material de Apoio VIII: Viewing 2D - Conceitos e Atividades

Monitor(a): Felipe Coelho
Profª Responsável: Luana Batista



Sumário

1.	Introdução	3
1.1.	Viewing Pipeline 2D	3
1.2.	Janela de Recorte ou Clipping Window	3
1.3.	Viewport	3
1.4.	Transformação 2D da Visualização	4
2.	Janela de Recorte	5
3.	Algoritmos de Recorte: Recorte de Ponto 2D, Linha 2D e Polígono 2D	7
3.1.	Algoritmos de Recorte	7
3.2.	Recorte de Ponto 2D	8
3.3.	Recorte de Linha 2D	8
3.4.	Recorte de Polígonos 2D	15
4.	Programação OpenGL: Múltiplas Viewports	20
4.1.	Janela de Recorte (Clipping Window)	20
4.2.	Viewport (Área de Exibição)	21
4.3.	Uso de Múltiplas Viewports	22
4.4.	Exemplo Completo de Uso de Viewports e Janela de Recorte	23
5.	Atividades	25

1 - Introdução

1.1 Viewing Pipeline 2D:

Processo responsável por gerar a **visualização bidimensional** de uma cena, definindo quais elementos serão exibidos e suas respectivas posições na tela. A imagem é inicialmente representada no **sistema de coordenadas do mundo (*world coordinates*)**, e determinadas partes selecionadas são transformadas para o **sistema de coordenadas do dispositivo (*device coordinates*)**. Esse processo envolve operações como **translação, rotação e escalonamento**, além da remoção de regiões que ficam fora da área visível.

1.2 Janela de Recorte ou Clipping Window:

Região específica de uma cena 2D escolhida para exibição. Perceba que qualquer elemento fora dessa região é essencialmente descartado.

1.3 Viewport:

A janela de recorte (**Figura 1**) pode ser inserida dentro de uma janela do sistema por meio de outra região chamada Viewport. Os objetos visíveis dentro da Janela de Recorte são ajustados para a Viewport, que, por sua vez, é posicionada dentro da interface do sistema. É possível utilizar múltiplas Viewports para exibir diferentes partes da imagem em posições variadas.

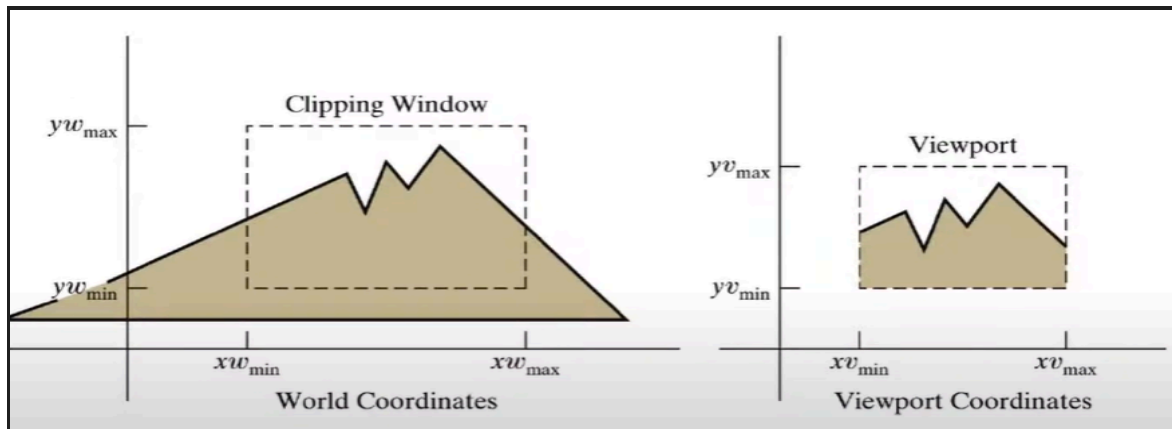


Figura 1

1.4 Transformação 2D da Visualização:

Conversão de uma representação da cena do sistema de coordenadas do mundo para o sistema de coordenadas do dispositivo (**Figura 2**).

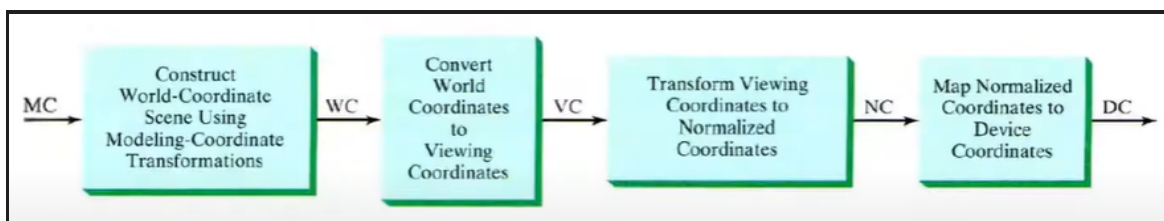


Figura 2

Para garantir que o processo seja compatível com diferentes dispositivos de saída, os sistemas gráficos convertem a representação dos objetos para um sistema de coordenadas normalizado, geralmente variando entre 0 e 1 ou entre -1 e 1.

2 - Janela de Recorte

Embora seja possível definir janelas de recorte em diversos formatos, a maioria das APIs gráficas suporta apenas janelas retangulares alinhadas aos eixos x e y , devido às restrições de custo computacional. No OpenGL, a janela de recorte é definida no sistema de coordenadas do mundo.

Quando a viewport está definida no intervalo de 0 a 1, é necessário transformar a representação dos objetos para esse espaço normalizado, preservando a posição relativa dos pontos conforme estabelecido na janela de recorte (**Figura 3**).

O ponto (xw, yw) é convertido para (xv, yv) conforme indicado abaixo :

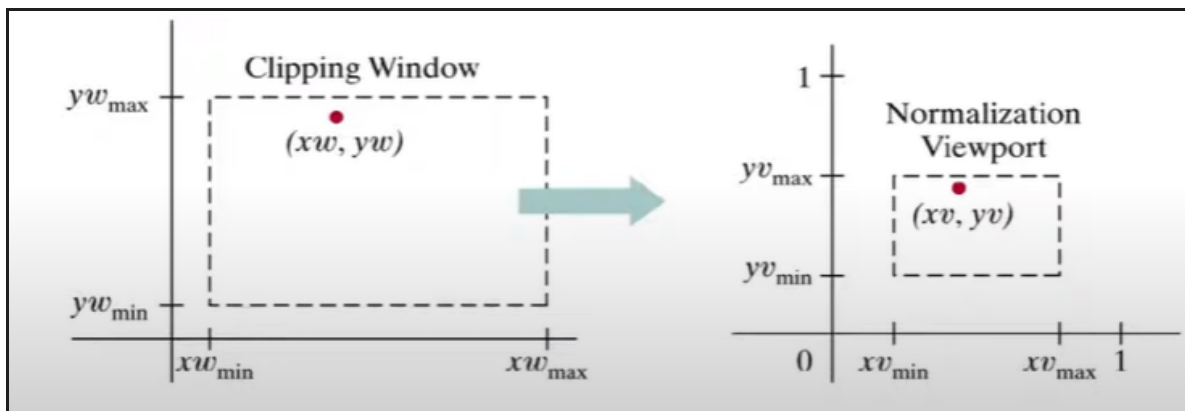


Figura 3

Para converter um ponto do sistema de coordenadas do mundo para a viewport, é necessário aplicar a seguinte transformação (**Figura 4**):

$$\frac{xv - xv_{min}}{xv_{max} - xv_{min}} = \frac{xw - xw_{min}}{xw_{max} - xw_{min}}$$

$$\frac{yv - yv_{min}}{yv_{max} - yv_{min}} = \frac{yw - yw_{min}}{yw_{max} - yw_{min}}$$

Figura 4

Nesse mapeamento, as posições relativas dos objetos permanecem inalteradas. Um objeto que estiver dentro da janela de recorte também aparecerá dentro da viewport.

As proporções dos objetos serão preservadas apenas se a relação de aspecto da viewport for igual à da janela de recorte.

O passo final é ajustar a posição da viewport dentro da janela da tela, conforme ilustrado na imagem abaixo (**Figura 5**):

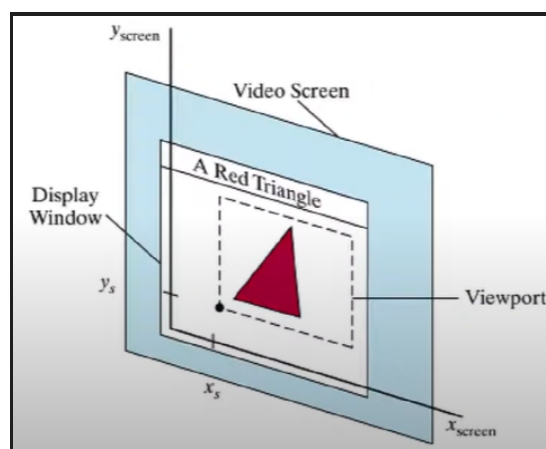


Figura 5

3 - Algoritmos de Recorte: Recorte de Ponto 2D, Linha 2D e Polígonos 2D

3.1 Algoritmos de Recorte:

No processo de visualização (**Viewing Pipeline**), o recorte é utilizado para selecionar uma parte específica da cena que será exibida no dispositivo de saída. Esse processo identifica e descarta as regiões da imagem que estão fora da janela de recorte, garantindo que apenas os elementos desejados sejam processados e exibidos.

Para otimizar o desempenho, o recorte é realizado sobre janelas de recorte normalizadas. Isso reduz a quantidade de cálculos, pois todas as transformações geométricas e de visualização podem ser combinadas em uma única matriz antes da execução do recorte.

Existem diferentes algoritmos para o recorte de:

- Pontos
- Linhas (segmentos retos)
- Regiões preenchidas (polígonos)
- Curvas
- Texto

Os três primeiros são amplamente utilizados em bibliotecas gráficas. O processamento é mais eficiente quando as bordas dos objetos são compostas por segmentos de reta.



Nas futuras análises a seguir, considere que a região de recorte corresponde a uma janela retangular posicionada de forma padronizada, com fronteiras definidas por $XWmin$, $XWmax$, $YWmin$ e $YWmax$. Geralmente, essa região corresponde a um espaço normalizado entre 0 e 1 ou -1 e 1.

3.2 Recorte de Ponto 2D:

Dado um ponto $P(x,y)$, ele será exibido no dispositivo de saída se, e somente se:

$$XWmin \leq XWmax$$

$$YWmin \leq YWmax$$

Esse método é especialmente útil para representar sistemas de partículas, como fumaça, nuvens e explosões.

3.3 Recorte de Linha 2D:

Cada linha da cena passa por testes e cálculos de interseção para determinar quais partes precisam ser desenhadas.

A operação mais custosa nesse processo é a determinação dos pontos de interseção das linhas com a janela de recorte. Por isso, o objetivo principal é minimizar esses cálculos sempre que possível.

A verificação se uma linha está inteiramente dentro da janela é simples, mas determinar se ela está completamente fora é mais desafiador:

- Se os dois pontos extremos de uma linha estiverem dentro da janela (exemplo: linha P1P2), a linha inteira será exibida.
- Se ambos os pontos estiverem fora da mesma borda da janela (linha P3P4), a linha será totalmente descartada.

- Caso nenhum desses critérios seja atendido, a linha cruza pelo menos uma das bordas da janela e pode ou não passar pelo seu interior, exigindo um cálculo mais detalhado (**Figura 6**).

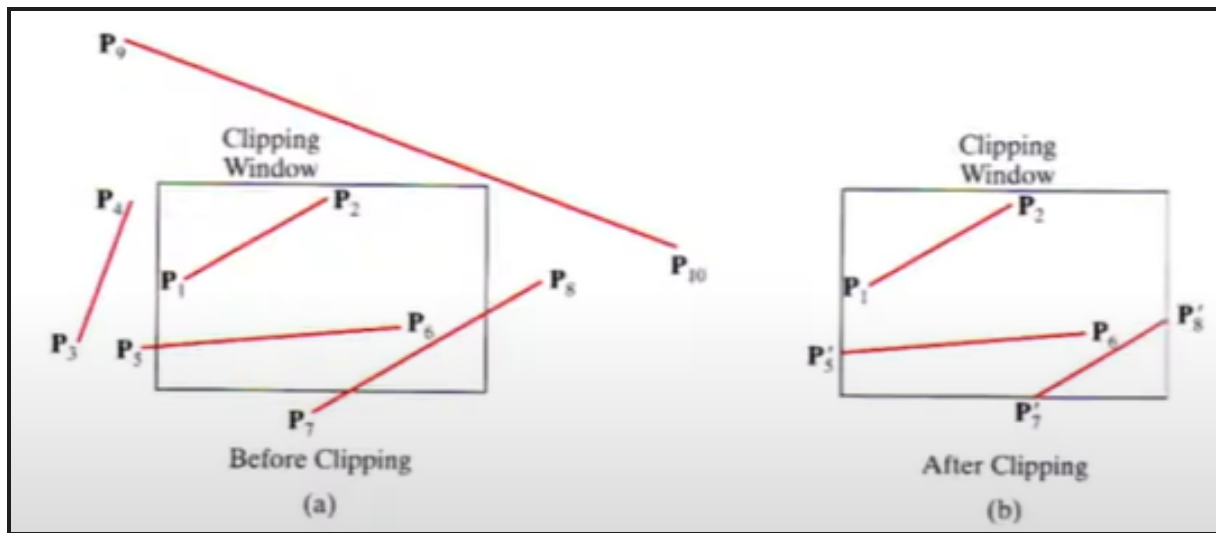


Figura 6

3.3.1 Algoritmo de Recorte de Cohen-Sutherland:

Um dos algoritmos pioneiros para otimizar o processo de recorte, reduzindo o tempo necessário ao realizar testes preliminares antes de calcular as interseções.

Inicialmente, cada extremidade das linhas recebe um código binário de 4 bits (**Figura 7**), conhecido como código de região.

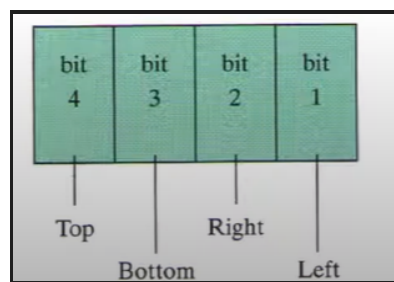


Figura 7

Os códigos binários indicam se um ponto está além de uma determinada fronteira:

- **0 (falso):** O ponto está dentro ou exatamente sobre a fronteira.
- **1 (verdadeiro):** O ponto está fora da fronteira.

A combinação dessas quatro fronteiras divide o espaço em nove regiões distintas (**Figura 8**), como ilustrado a seguir:

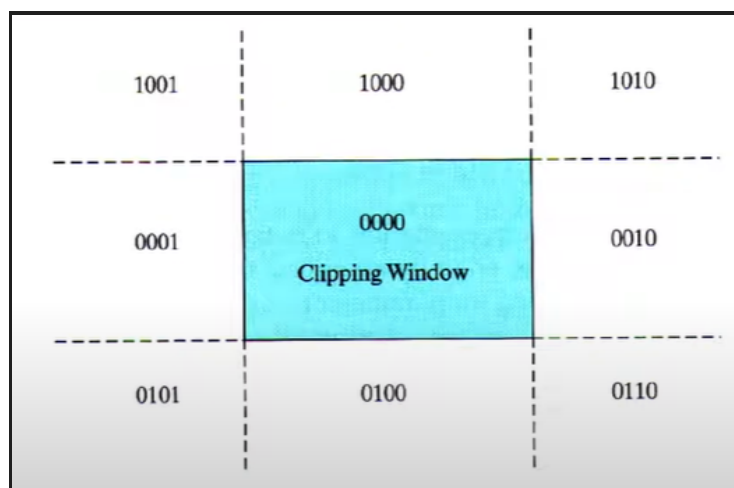


Figura 8

Um ponto localizado abaixo e à esquerda da janela de recorte recebe o código **0101**, enquanto um ponto dentro da janela recebe **0000**.

Os valores dos bits são atribuídos comparando as coordenadas (x,y) do ponto com as fronteiras da janela de recorte:

- **Bit 4 é 1** se $Y > YW_{max}$ (fora da borda superior (T)).
- **Bit 3 é 1** se $Y < YW_{min}$ (fora da borda inferior (B)).
- **Bit 2 é 1** se $X > XW_{max}$ (fora da borda direita (R)).
- **Bit 1 é 1** se $X < XW_{min}$ (fora da borda esquerda (L)).

Com esses códigos, é possível identificar rapidamente se uma linha está totalmente dentro ou fora da janela de recorte:

- Linhas inteiramente dentro da janela possuem ambos os pontos com código **0000**.
- Linhas cujos pontos finais compartilham **1s** nas mesmas posições estão completamente fora da área visível.
- Se os pontos finais de uma linha possuem códigos **1001** e **0101**, a linha está totalmente à esquerda da janela de recorte.

Linhas que não podem ser classificadas imediatamente como dentro ou fora da janela são então processadas para calcular suas interseções com as fronteiras da área visível (**Figura 9**).

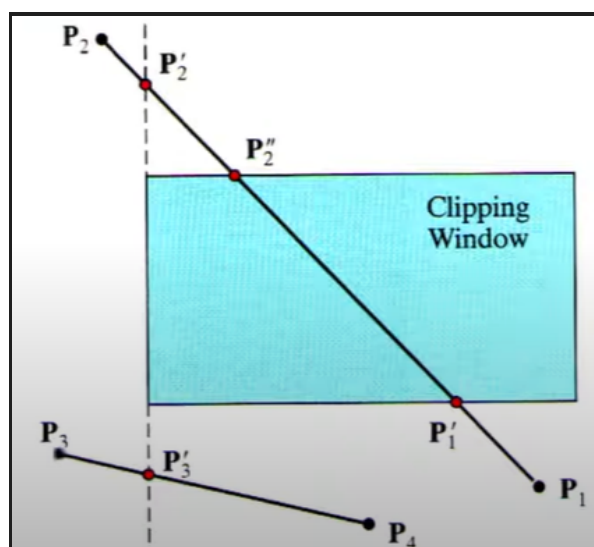


Figura 9

À medida que cada interseção com os limites da janela de recorte é determinada, a linha é ajustada, preservando apenas a parte que permanece dentro da janela ou eliminando-a por completo caso nenhum trecho esteja contido nela.

Para verificar se uma linha cruza um dos limites **(Figura 10)**, basta analisar os bits correspondentes às fronteiras nos pontos terminais. Se um dos bits for 1 e o outro 0, significa que a linha atravessa a fronteira.

Iniciando pelo limite esquerdo:

$P_1 = 0100 \rightarrow$ está dentro da fronteira esquerda

$P_2 = 1001 \rightarrow$ está fora da fronteira esquerda

Calcula-se a interseção P'_2 e remove-se o segmento $P_2P'_2$.

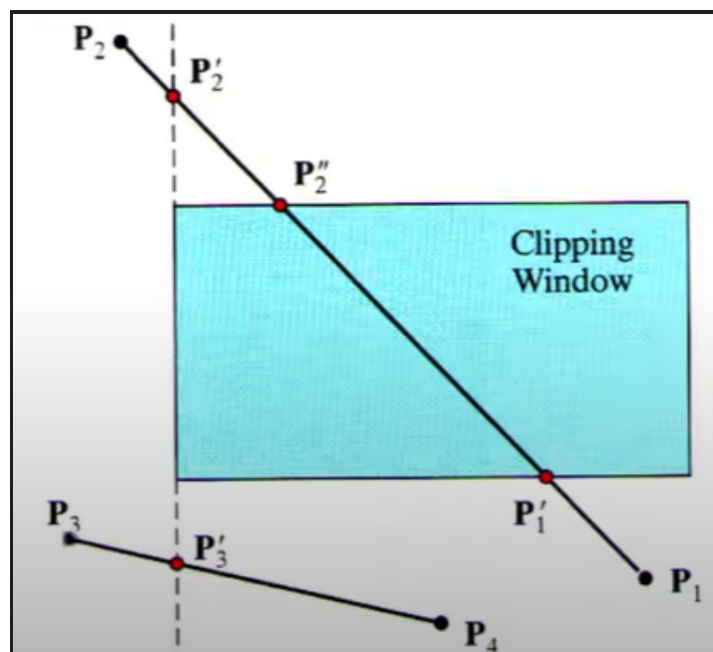


Figura 10

As demais bordas (direita, inferior e superior) seguem a mesma lógica.



Para calcular os valores de Y nas interseções da linha definida pelos pontos (X_o, Y_o) e (X_{end}, Y_{end}) com as fronteiras verticais (esquerda e direita), utilizamos a equação explícita:

$$Y = Y_o + m(X - X_o)$$

Onde o valor de X será $XWmin$ (fronteira esquerda) ou $XWmax$ (fronteira direita), e a inclinação da reta é determinada por:

$$m = (Y_{end} - Y_o) / (X_{end} - X_o)$$

Para encontrar os valores de X nas interseções da linha com as fronteiras horizontais (superior e inferior), utilizamos:

$$X = X_o + ((Y - Y_o) / m)$$

Já aqui, Y assume os valores de $YWmin$ (fronteira inferior) ou $YWmax$ (fronteira superior).

3.3.2 Exemplo prático:

Considere uma janela de recorte com as dimensões:

- $XWmin = 100, XWmax = 300$
- $YWmin = 150, YWmax = 250$

E uma linha definida pelos pontos:

- $P_o = (280, 225)$
- $P_{end} = (350, 160)$

Aplicamos o algoritmo de Cohen-Sutherland para o recorte de linhas 2D.

Atribuição de códigos binários

Os códigos seguem a regra: **0** → **Dentro**, **1** → **Fora**

- $P_o \rightarrow (TBRL) = (0000)$
- $P_{end} \rightarrow (TBRL) = (0010)$

Analisando os códigos:

- A linha não está completamente contida dentro da janela, pois ambos os códigos deveriam ser **0000**.
- Não há **1s** nas mesmas posições dos dois pontos finais, indicando que a linha também não está totalmente fora da janela.
- A linha cruza a **fronteira direita**, pois o bit correspondente em P_o é **0**, enquanto em P_{end} é **1**.

Cálculo das coordenadas de P'o (Figura 11):

Sabemos que $X = XW_{max} = 300$, então:

$$P'o = (300, y)$$

Para encontrar Y:

$$m = (Y_{end} - Y_o) / (X_{end} - X_o) = 160 - 225 / 350 - 280$$

$$m = -65/70 \approx -0,9285$$

$$Y = Y_o + m(X - X_o) = 225 + (-0,9285) \times (300 - 280)$$

$$Y = 225 - 18,57 = 206,43$$

Portanto, a nova coordenada do ponto ajustado é: $P'o = (300, 206,43)$

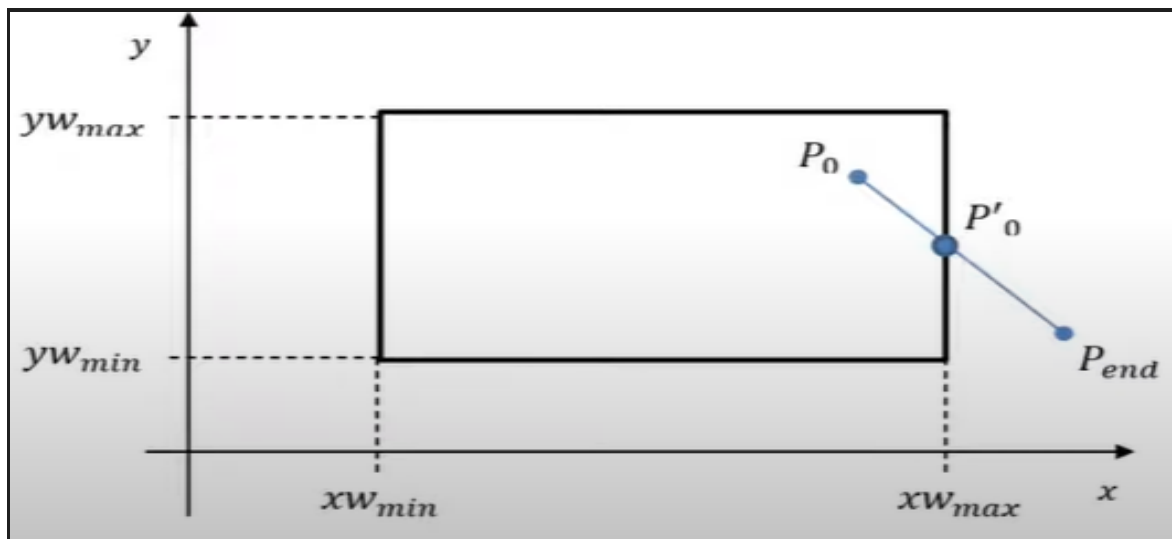


Figura 11

3.4 Recorte de Polígonos 2D:

Os algoritmos de recorte de linhas não são adequados para o corte de polígonos, pois, em geral, não garantem a preservação da forma fechada do polígono. Em vez disso, resultam em segmentos de linha desconectados (**Figura 12**), sem qualquer indicação de como reuni-los para reconstruir o polígono recortado corretamente.

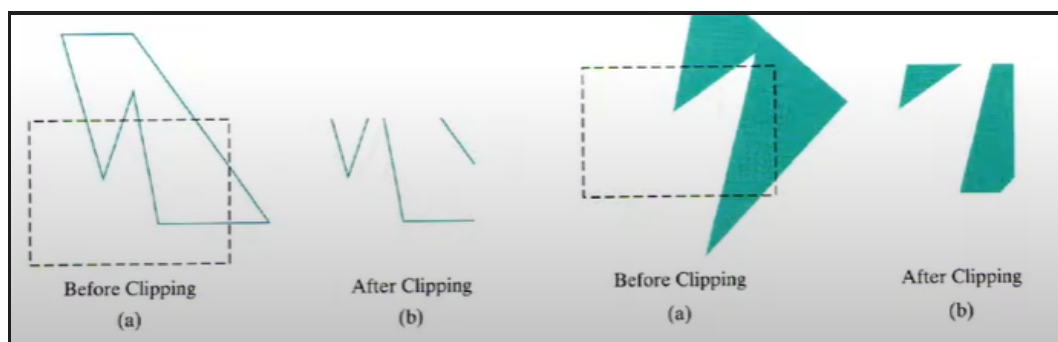


Figura 12

É possível realizar o processamento do polígono em relação às bordas da janela de recorte (**Figura 13**) de maneira semelhante ao recorte de linhas. Isso é feito redefinindo a forma do polígono a cada vez que uma das fronteiras da janela é analisada.

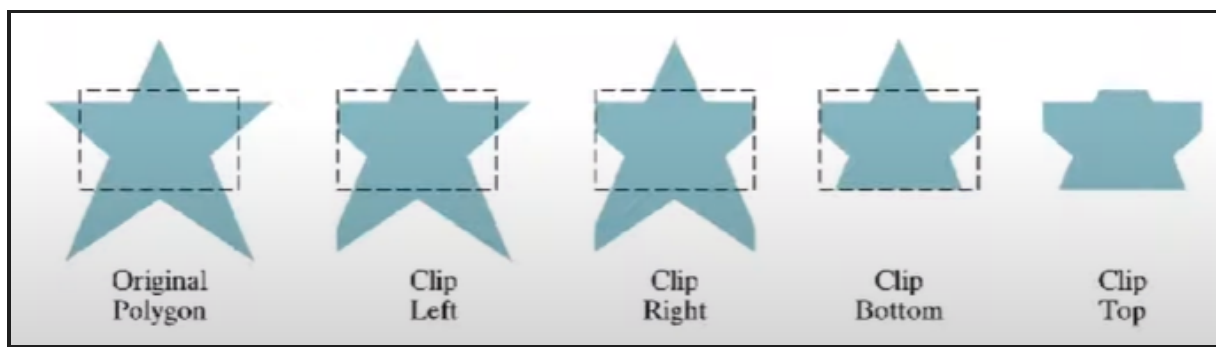


Figura 13

Podemos determinar se um polígono está totalmente dentro ou fora da janela de recorte analisando suas coordenadas mínimas e máximas (**Figura 14**). Caso a região não possa ser classificada como inteiramente dentro ou fora, é necessário calcular as interseções.

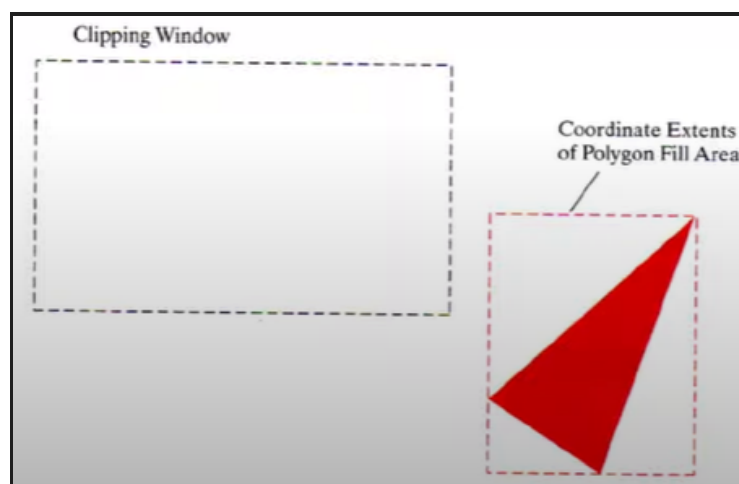


Figura 14

Uma abordagem simples para recortar polígonos convexos **(Figura 15)** é gerar uma nova lista de vértices após cada recorte contra uma fronteira e utilizá-la como base para o próximo recorte em outra borda. No caso de polígonos côncavos, o processo se torna mais complexo, podendo levar à criação de múltiplas listas de vértices.

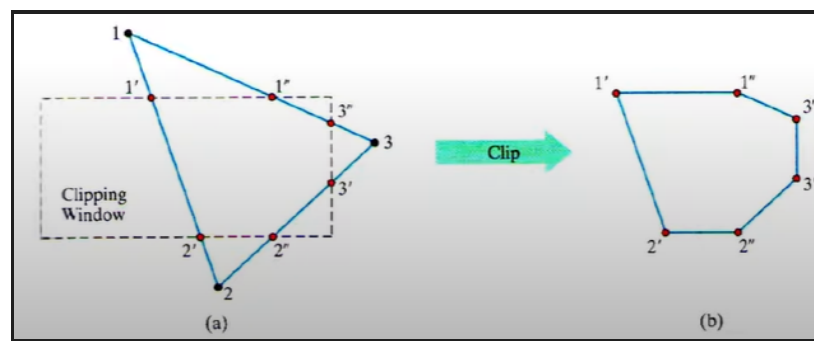


Figura 15

3.4.1 Algoritmo de Sutherland-Hodgman:

Uma maneira eficiente de recortar polígonos é processar seus vértices em diferentes etapas de recorte, garantindo que os vértices ajustados sejam imediatamente utilizados na próxima fase.

Esse algoritmo funciona enviando os pares de pontos finais de cada aresta do polígono para uma sequência de recortes (esquerda, direita, inferior e superior). À medida que o recorte é aplicado a um par de vértices, os pontos resultantes são encaminhados para o próximo estágio do processo.

As arestas do polígono são analisadas individualmente e processadas em ordem anti-horária para cada uma das bordas da janela de recorte.

3.4.2 Casos possíveis durante o processamento de uma aresta:

1. O primeiro vértice está fora da janela e o segundo dentro.
2. Ambos os vértices estão dentro da janela.
3. O primeiro vértice está dentro e o segundo fora.
4. Ambos os vértices estão fora da janela.

3.4.3 Regras para o envio dos vértices entre os recortadores:

Cada par de vértices sucessivos passa por um dos recortadores e, conforme sua posição em relação à janela, a saída é gerada para o próximo estágio da seguinte forma (**Figura 16 e Figura 17**):

1. Se o primeiro vértice está fora e o segundo dentro, a interseção calculada e o segundo vértice são enviados.
2. Se ambos os vértices estão dentro, apenas o segundo vértice é encaminhado.
3. Se o primeiro vértice está dentro e o segundo fora, somente a interseção é enviada.
4. Se ambos os vértices estão fora, nada é transmitido.

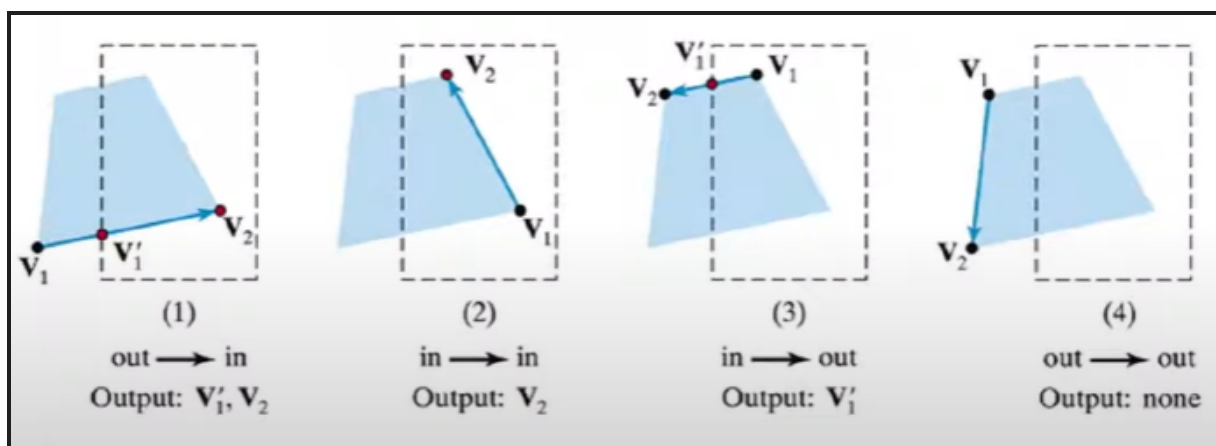


Figura 16

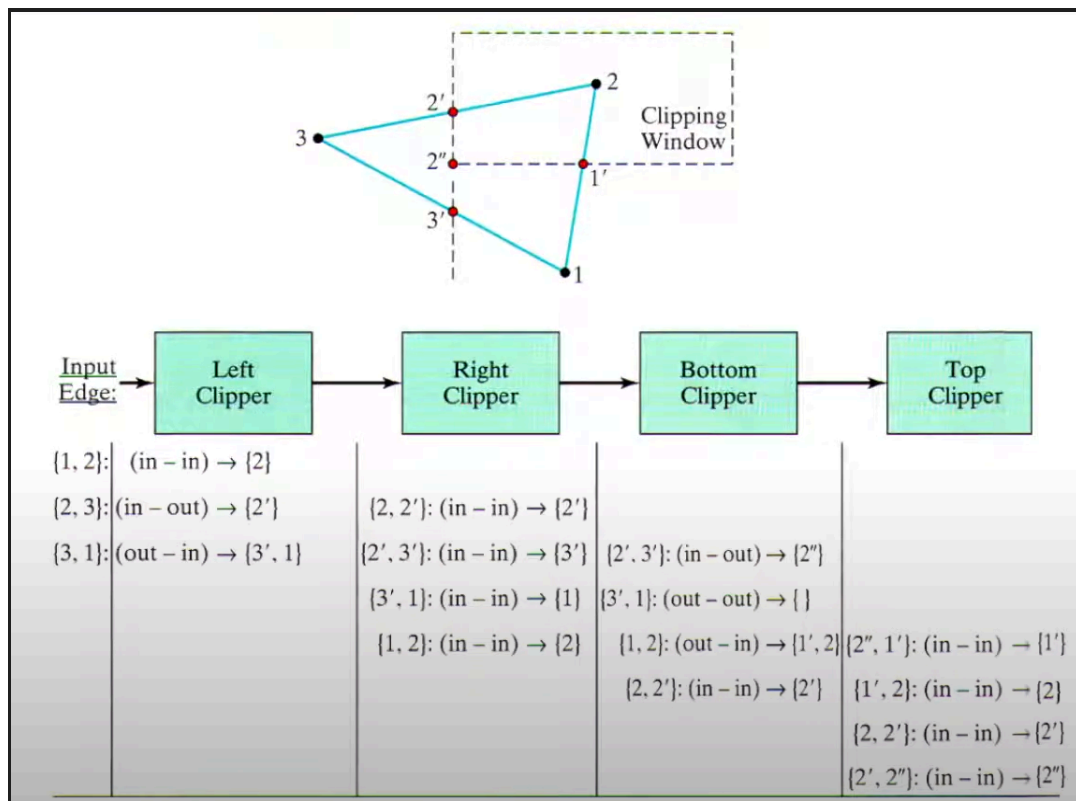


Figura 17

No entanto, existe uma limitação: para polígonos côncavos, podem surgir problemas, pois esse algoritmo gera apenas uma única lista de vértices como resultado. Uma solução (**Figura 18**) seria dividir o polígono côncavo em partes convexas.

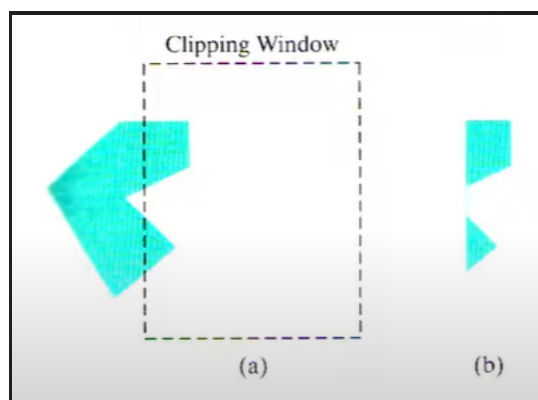


Figura 18

4 - Programação OpenGL: Múltiplas Viewports

No OpenGL, podemos definir diferentes viewports dentro da mesma janela de exibição, o que é útil para dividir a tela em múltiplas áreas de visualização, como em aplicações de visualização científica, jogos e CAD (Computer-Aided Design).

Antes de configurar a janela de recorte e a viewport, é essencial definir que a matriz ativa é a matriz de projeção. Para isso, utilizamos:

```
glMatrixMode(GL_PROJECTION);
```

Lembre-se de que as transformações em OpenGL são acumulativas. Portanto, para evitar efeitos inesperados, deve-se redefinir a matriz para sua forma original usando

```
glLoadIdentity();
```

4.1 Janela de Recorte (Clipping Window)

A janela de recorte define a região do mundo que será projetada na tela e pode ser configurada com o seguinte comando:

```
gluOrtho2D(GLfloat XWmin, GLfloat XWmax, GLfloat YWmin, GLfloat YWmax);
```

Neste caso:

- $XWmin, XWmax$ → delimitam os limites horizontais mínimo e máximo do espaço de coordenadas do mundo.
- $YWmin, YWmax$ → definem os limites vertical mínimo e máximo.

IMPORTANTE

Se a janela de recorte não for especificada, o OpenGL adotará os seguintes valores padrão:

- $XWmin = YWmin = -1$
- $XWmax = YWmax = +1$

Isso significa que o processo de recorte ocorrerá dentro de um espaço normalizado no intervalo $[-1, 1][-1, 1]$.

4.2 Viewport (Área de Exibição)

A viewport define a área dentro da janela onde o conteúdo renderizado será exibido. Para configurá-la, utilizamos:

`glViewport(GLint XVmin, GLint YVmin, GLsizei vpWidth, GLsizei vpHeight);`

Onde:

- $XVmin, YVmin$ → coordenadas do canto inferior esquerdo da viewport em pixels.
- $vpWidth, vpHeight$ → largura e altura da viewport, respectivamente.



Se a função **glViewport(...)** não for chamada, a viewport assumirá o tamanho total da janela de exibição.

4.3 Uso de Múltiplas Viewports:

Podemos definir várias viewports dentro de uma mesma janela para exibir diferentes perspectivas ou subdivisões da cena. Isso é feito chamando `glViewport(...)` várias vezes no mesmo programa.

Exemplo de divisão da tela em duas viewports verticais:

// Primeira viewport à esquerda

glViewport(0, 0, windowWidth / 2, windowHeight);

desenharCena();

// Segunda viewport à direita

glViewport(windowWidth / 2, 0, windowWidth / 2, windowHeight);

desenharOutraCena();

Isso cria uma divisão onde a primeira metade da janela exibe uma cena e a segunda metade exibe outra, permitindo comparações ou múltiplas visões de um mesmo modelo.

4.4 Exemplo Completo de Uso de Viewports e Janela de Recorte (Figura 19):

```
1 #include <GL/glut.h>
2
3 void desenharCena() {
4     glClear(GL_COLOR_BUFFER_BIT);
5
6     // Definir a matriz de projeção
7     glMatrixMode(GL_PROJECTION);
8     glLoadIdentity();
9     gluOrtho2D(-10, 10, -10, 10);
10
11     // Primeira viewport (metade esquerda da janela)
12     glViewport(0, 0, 250, 500);
13     glColor3f(1, 0, 0); // Cor vermelha
14     glBegin(GL_POLYGON);
15         glVertex2f(-5, -5);
16         glVertex2f(5, -5);
17         glVertex2f(5, 5);
18         glVertex2f(-5, 5);
19     glEnd();
20
21     // Segunda viewport (metade direita da janela)
22     glViewport(250, 0, 250, 500);
23     glColor3f(0, 0, 1); // Cor azul
24     glBegin(GL_TRIANGLES);
25         glVertex2f(-5, -5);
26         glVertex2f(5, -5);
27         glVertex2f(0, 5);
28     glEnd();
29
30     glFlush();
31 }
32
33 int main(int argc, char** argv) {
34     glutInit(&argc, argv);
35     glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
36     glutInitWindowSize(500, 500);
37     glutCreateWindow("Múltiplas Viewports");
38
39     glClearColor(1, 1, 1, 1);
40     glutDisplayFunc(desenharCena);
41
42     glutMainLoop();
43     return 0;
44 }
```

Figura 19

Explicação do Código:

- Criamos uma **janela de 500x500 pixels**.
- Definimos uma **janela de recorte ortogonal** no espaço de coordenadas do mundo.
- Criamos **duas viewports**, dividindo a tela verticalmente.

- Desenhamos um **quadrado vermelho na esquerda** e um **triângulo azul na direita**.
- Configuramos a cor de fundo como branco.
- Iniciamos o **loop principal** para manter a janela aberta e responder a eventos.

Esse código pode ser expandido para incluir mais viewports, animações ou interatividade.

Isso demonstra como utilizar múltiplas viewports em OpenGL para exibir diferentes elementos gráficos em uma única janela (**Figura 20**). Abaixo segue o resultado:

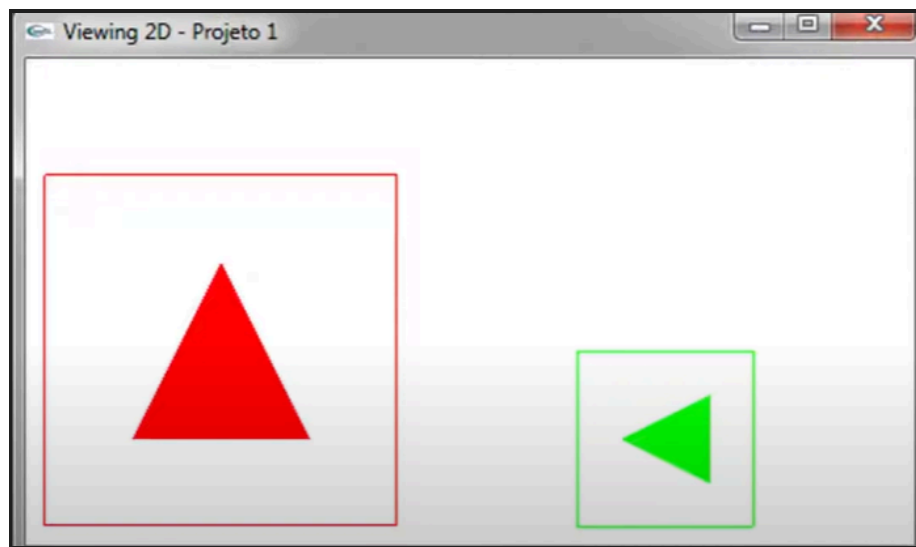


Figura 20

Logo abaixo segue uma lista de exercícios para se aprofundar e revisar todo o conteúdo, qualquer dúvida a respeito pode entrar em contato com os monitores via email ou chamada meet nos horários pré estabelecidos.

5 - Atividades

Parte 1 - Questões Teóricas

1. Defina o conceito de **Viewing Pipeline 2D** e explique sua importância no processamento gráfico.
2. Diferencie **janela de recorte (clipping window)** e **viewport**.
3. Explique como ocorre a transformação de um ponto do sistema de coordenadas do mundo para a viewport.
4. Por que a maioria das **APIs gráficas** utilizam janelas de recorte retangulares alinhadas aos eixos **x** e **y**?
5. Descreva as principais etapas do **algoritmo de Cohen-Sutherland** para **recorte de linhas 2D**.
6. Qual a diferença entre os algoritmos de **recorte de linha** e os de **recorte de polígonos**?
7. Explique a limitação do **algoritmo de Sutherland-Hodgman** ao **recortar polígonos côncavos** e sugira uma possível solução.

Parte 2 - Exercícios Práticos

1. Transformação de Coordenadas

Implemente um programa que:

- Receba como entrada um ponto (x_w, y_w) no sistema de coordenadas do mundo e uma janela de recorte definida por ($XW_{min}, XW_{max}, YW_{min}, YW_{max}$).

- Mapeie esse ponto para o sistema de coordenadas da viewport normalizado entre $[0, 1]$ considerando uma viewport definida por ($XVmin = 0$, $XVmax = 1$, $YVmin = 0$, $YVmax = 1$).
- Exiba o ponto transformado (xv , yv).

2. Implementação do Algoritmo de Cohen-Sutherland

Implemente o algoritmo de Cohen-Sutherland para recortar linhas dentro de uma janela de recorte retangular.

O programa deve:

- Receber como entrada os pontos $P1(x1, y1)$ e $P2(x2, y2)$ de uma linha e os limites da janela de recorte.
- Determinar se a linha está completamente dentro, completamente fora ou se precisa ser recortada.
- Calcular e exibir os pontos resultantes após o recorte.

3. Implementação do Algoritmo de Sutherland-Hodgman

Implemente o algoritmo de Sutherland-Hodgman para recortar polígonos dentro de uma janela de recorte retangular. O programa deve:

- Receber como entrada um conjunto de vértices de um polígono e os limites da janela de recorte.
- Determinar a nova forma do polígono após o recorte.
- Exibir os vértices do polígono resultante.



4. Múltiplas Viewports em OpenGL

Crie um programa em OpenGL que utilize múltiplas viewports para exibir diferentes seções de uma mesma cena. O programa deve:

- Criar pelo menos duas viewports distintas dentro da janela gráfica.
- Exibir diferentes partes de uma mesma cena em cada viewport.
- Utilizar `glMatrixMode(GL_PROJECTION)` e `gluOrtho2D()` para definir as janelas de recorte.